

وزارة التربية والتعليم العالي
مديرية التربية والتعليم: جنوب الخليل
المدرسة: ذكور دورا الثانوية المهنية
التخصص: تطبيقات الهواتف الذكية

عنوان المشروع:
جذور فلسطين

اسم الطالب: منتصر عبد الكريم رجوب
رقم الجلوس: 27057314
المعلم المشرف: م. مؤيد حريبات

قدم هذا المشروع للحصول على شهادة الكفاءة المهنية

2025 / 2026

1	الفهرس	2
2	شكر وعرفان	3
3	ملخص المشروع	4
4	فكرة المشروع والمحتوى التعليمي	5
5	مكونات المشروع	6
6	خطة المشروع	7
7	تنفيذ المشروع	8
8	واجهات التصميم	9-11
9	التوصيات	12
10	الصعوبات والتحديات	13
11	الخلاصة	14
12	المراجع والمصادر	15
13	الملاحق (الشفرات المصدريّة)	16-22

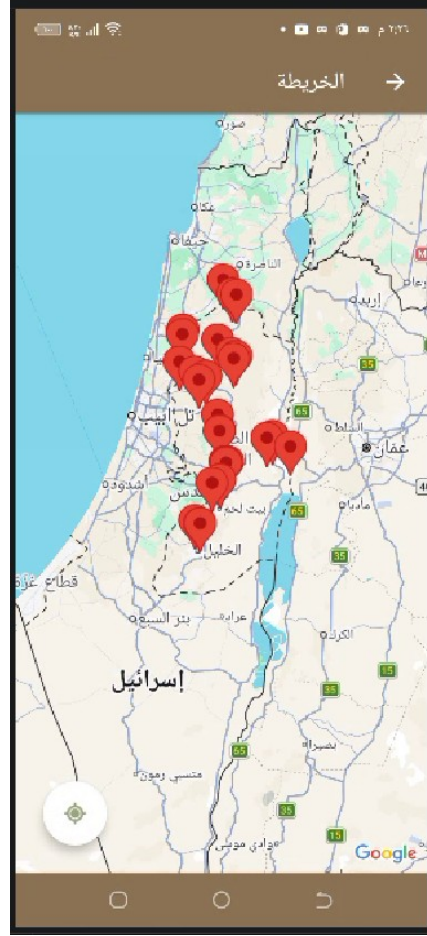
شكر وعرفان

بسم الله الرحمن الرحيم، الحمد لله الذي بنعمته تتم الصالحات، والصلاة والسلام على سيدنا محمد معلم البشرية.

أتقدم بخالص الشكر والتقدير إلى مشرفي المهندس مؤيد حريبات، الذي لم يخل علي بتوجيهاته وخبرته، وكان لصبره وسعة صدره الأثر الكبير في تذليل الصعوبات التقنية وإنجاز هذا العمل. كما أتوجه بالشكر إلى إدارة ومعلمي مدرسة ذكور دورا الثانوية المهنية على تهيئة البيئة الداعمة للإبداع التقني، وإلى والدي على دعمهما المستمر، وإلى زملائي الذين كانوا عوناً لي خلال مراحل التطوير. وأخص بالشكر كل من ساهم في نجاح هذا المشروع ولو بكلمة توجيه أو دعم معنوي.

ملخص المشروع

يُعد تطبيق "جذور فلسطين" منصة رقمية تفاعلية تهدف إلى توحيد وتوثيق المعلومات الخاصة بالمواقع الأثرية والتراثية في الضفة الغربية، والتي كانت مُشتتة سابقاً بين مصادر متعددة يصعب الوصول إليها. يعتمد التطبيق تجمع بين كفاءة التخزين المحلي، وسلاسة واجهة (Clean Architecture) على معمارية برمجية حديثة لضمان استمرار الخدمة حتى في Offline-First المستخدم، وتكامل الخرائط التفاعلية. يعمل بنظام المناطق ضعيفة الاتصال، ويدعم لغتين (عربية وإنجليزية) مع وضع ليلي مريح للعين. يجمع المشروع بين الدقة الهندسية والقيمة الثقافية، مقدماً تجربة استكشاف رقمية متكاملة للسياح، الطلاب، والمهتمين بالتراث الفلسطيني.



رسم توضيحي 1: صفحة التحميل

فكرة المشروع والمحتوى التعليمي

الفكرة

توفير قاعدة بيانات محلية شاملة ومحدثة لـ 30 موقعاً أثرياً ودينيّاً موزعة على 10 محافظات فلسطينية، مع واجهة سهلة الاستخدام تتيح البحث، الفلترة، التنقل الخرائطي، وحفظ المواقع المفضلة، كل ذلك دون الحاجة لاتصال دائم بالإنترنت

المحتوى التعليمي والمهارات المكتسبة

- لفصل طبقات البيانات والمنطق والواجهة MVVM و Clean Architecture تطبيق نمط
- للتخزين الهيكلي وإدارة العمليات غير المتزامنة Room Database استخدام
- لضمان الأمان والاستقرار ViewBinding بالكود عبر XML ربط واجهات
- Flow و Coroutines إدارة حالة التطبيق وتدفق البيانات باستخدام
- وعرض الإحداثيات الجغرافية بدقة Google Maps SDK دمج
- ودعم الوضع الليلي/النهاري Material Components تطبيق
- لضمان ViewModelProvider.Factory باستخدام (Manual DI) الحقن اليدوي للتبعيات
- قابلية الاختبار والصيانة
- مع حفظ التفضيل عبر LocaleHelper دعم تعدد اللغات (عربي/إنجليزي) باستخدام
- SharedPreferences.
- Kotlin وتحويلها إلى كائنات JSON لتحليل بيانات Gson استخدام
- مع مراقبة التغيرات عبر PreferencesManager عبر SharedPreferences استخدام
- Flow (callbackFlow).

مكونات المشروع

المكونات البرمجية

Kotlin : لغة البرمجة

Material Components و ViewBinding مع XML Views : واجهة المستخدم

Room Database : قاعدة البيانات المحلية

PreferencesManager عبر SharedPreferences : إدارة الإعدادات

Gson : معالجة البيانات

Flow و Coroutines : إدارة الخلفية

Glide : تحميل الصور

Google Maps SDK : الخرائط

ViewModelProvider.Factory عبر (Manual DI) حقن التبعيات: الحقن اليدوي

SharedPreferences مع LocaleHelper : دعم اللغات

المكونات البياناتية

- هيكلي يحتوي على 30 موقعاً (اسم، وصف، إحداثيات، تصنيف، محافظة) مع حقول ثنائية اللغة JSON ملف
- Assets مكتبة صور عالية الجودة مخزنة محلياً في مجلد
- Strings.xml نصوص ثنائية اللغة (عربي/إنجليزي) في

المكونات المعمارية

- Data Layer: Room, DAO, Repository, Gson Parser

- Domain Layer: Site Models, Repository Interfaces
- Presentation Layer: Activities, ViewModels, ViewBinding,
LocaleHelper

خطة المشروع

:تم تنفيذ المشروع على سبع مراحل رئيسية

المدة	الوصف	المرحلة
أسبوع	تحليل المتطلبات: جمع بيانات المواقع الأثرية، وتصميم هيكلية قاعدة البيانات	الأولى
أسبوع	تصميم واجهات المستخدم و XML باستخدام UI/UX Material Components	الثانية
أسبوع	إعداد بيئة التطوير، وربط المكتبات (Room, Coroutines, Gson, ViewBinding)	الثالثة
أسبوعان	برمجة طبقة البيانات، تنفيذ نمط وربط الواجهات، MVVM، بالمنطق البرمجي	الرابعة
أسبوعان	تفعيل، Google دمج خرائط البحث والفلتر، الوضع الليلي، نظام المفضلة، وتعدد اللغات	الخامسة
أسبوع	الاختبار الشامل على أجهزة حقيقية، إصلاح الأخطاء، وتحسين الأداء	السادسة
أسبوع	التوثيق الفني، إعداد التقرير النهائي، والتحضير للعرض أمام اللجنة	السابعة

:بخطوات منهجية (SDLC) تم تنفيذ المشروع وفق دورة حياة تطوير البرمجيات

1.

.بتسمية موحدة JSON تحليل البيانات: تجميع المعلومات التاريخية والجغرافية للمواقع، وهيكله ملف

2.

AGP 7.4.2، مع إصدارات مستقرة Gradle وربط، إعداد البيئة: تهيئة مشروع
JDK 11، compileSdk 33).

3.

assets، لقراءة البيانات الأولية من Gson وتفعيل، DAO واجهة، Room بناء طبقة البيانات: إنشاء كيان
بأسماء أعمدة قاعدة البيانات بدقة JSON لربط أسماء حقول @SerializedName مع استخدام

4.

لكل شاشة، وعزل المنطق عن العرض، وربط البيانات باستخدام ViewModel إنشاء **MVVM** تطبيق
ViewModelProvider.Factory مع استخدام الحقن اليدوي للتبعيات عبر، StateFlow

5.

ViewBinding وتفعيل، XML تطوير الواجهات: تصميم شاشات التطبيق بـ

6.

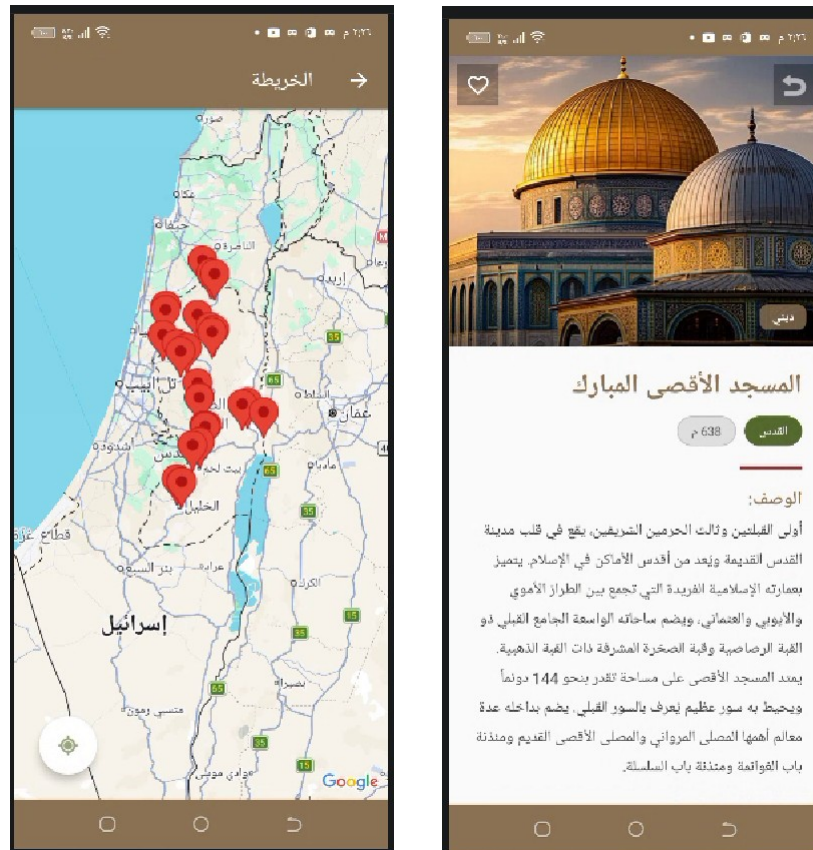
للعرض الجغرافي، تفعيل البحث والفلتر، نظام المفضلة عبر GoogleMap دمج الميزات المتقدمة: إضافة
LocaleHelper ودعم تعدد اللغات (عربي/إنجليزي) باستخدام، SharedPreferences

7.

.الاختبار والتحسين: التشغيل على أجهزة حقيقية، معالجة الأخطاء، وتحسين الأداء

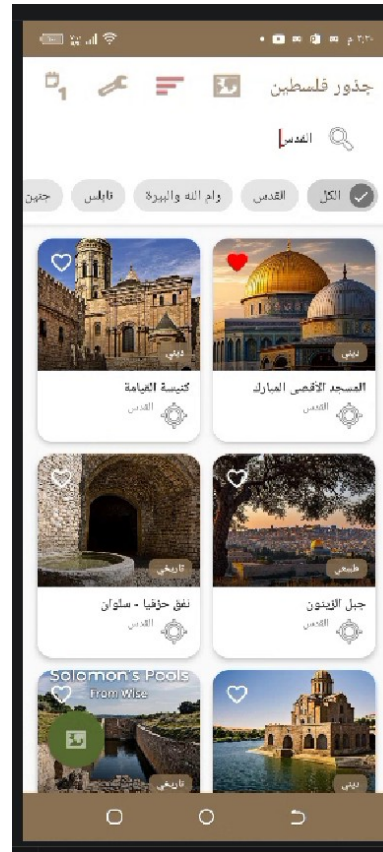
واجهات التصميم

:والشاشة الرئيسية (Splash) شاشة البداية



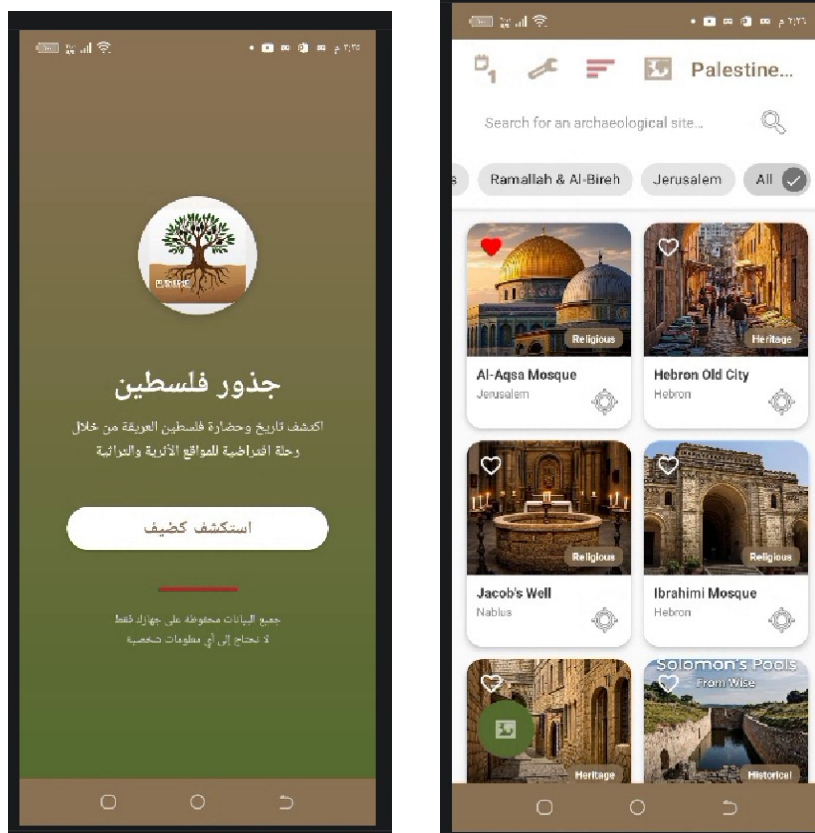
شاشة البداية والشاشة الرئيسية

:تسجيل الدخول والتعريف بالتطبيق



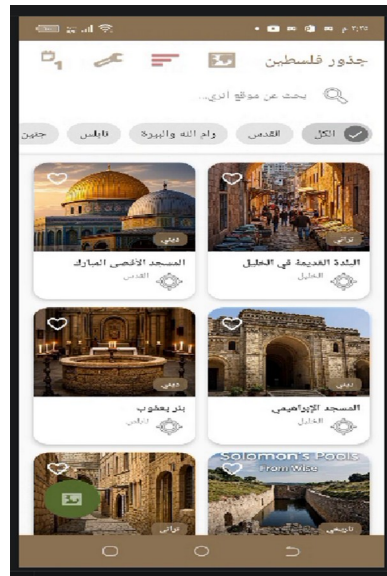
تسجيل الدخول والتعريف بالتطبيق

الخريطة التفاعلية والمفضلة



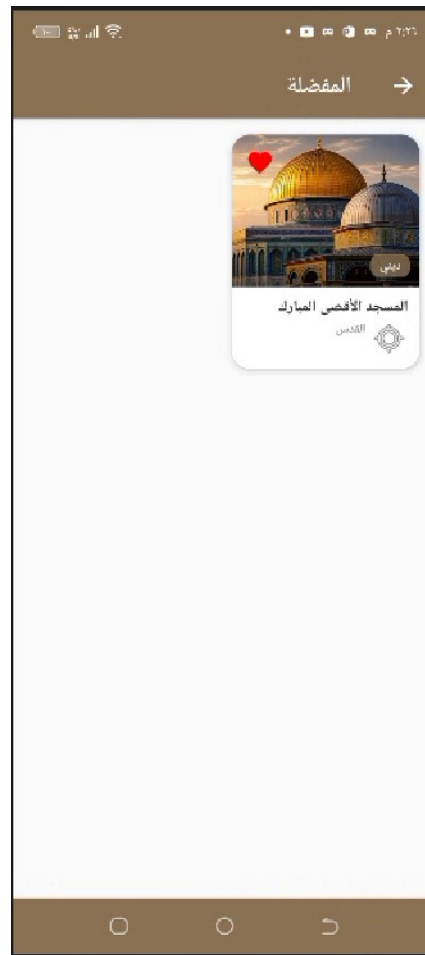
الخريطة التفاعلية والمفضلة

اللغة الإنجليزية والبحث



اللغة الإنجليزية والبحث

تفاصيل الموقع



شاشة تفاصيل الموقع

1. لتحديث البيانات تلقائياً عند توفر الإنترنت Firebase إضافة المزامنة السحابية: ربط التطبيق بـ.
2. لتجربة غامرة ARCore عرض نماذج ثلاثية الأبعاد للمواقع الأثرية باستخدام (AR): دعم الواقع المعزز.
3. نظام التقييم والتعليقات: تمكين المستخدمين من مشاركة تجاربهم وصورهم للمواقع.
4. توسيع نطاق اللغات: إضافة دعم للغات إضافية (فرنسية، إسبانية، عبرية) لتقديم التطبيق لشرح أوسع.
5. لتشجيع المطورين على المساهمة في تطوير GitHub نشر المشروع كمصدر مفتوح: رفع الكود على التطبيق.
6. تحسين نظام اللغات: إضافة المزيد من النصوص المترجمة وضمان توافق جميع الواجهات مع اتجاه النص (RTL/LTR).

الصعوبات والتحديات

:واجه المشروع عدة تحديات تقنية تطلبت حلولاً مبتكرة

1. Kotlin وكود JSON تعارض تسمية الحقول بين

الأثر: اختفاء بيانات المواقع عند التحميل

↔ imageUrl (مثل) لإنشاء ربط دقيق بين التسميتين @SerializedName الحل: استخدام
image_url, foundationYear ↔ foundation_year).

2. أعطال مفاجئة أثناء التنقل بين الشاشات

(Crash) الأثر: تعطيل التطبيق

والتأكد من استيراد Lifecycle، بشكل كامل مع إدارة دورة الحياة ViewBinding الحل: تفعيل
Context و Intent في جميع الأنشطة

3. تشتت إعدادات المستخدم

الأثر: فقدان التفضيلات عند إعادة التشغيل (اللغة، الوضع الليلي)

مع مراقبة المتغيرات عبر PreferencesManager عبر SharedPreferences الحل: استخدام
Flow (callbackFlow) لضمان استجابة فورية لتغيرات الإعدادات

4. توافق بيئة التطوير (JDK, Gradle, AGP)

الأثر: فشل عملية البناء

JDK 11, AGP 7.4.2, kotlinOptions و compileOptions الحل: توحيد الإصدارات عبر
للمعالجة مشكلة المسار android.overridePathCheck=true وإضافة (33 compileSdk),
العربي.

5. تحميل الصور من المسار المحلي

RAM الأثر: بطء القائمة وارتفاع استهلاك

.لتحسين أداء تحميل الصور وإدارتها بكفاءة Disk/Memory Cache مع Glide الحل: استخدام

دعم تعدد اللغات في وقت التشغيل 6.

الأثر: عدم استجابة التطبيق لتغيير اللغة بدون إعادة تشغيل

في كل نشاط، مع حفظ attachBaseContext لتطبيق اللغة عبر LocaleHelper الحل: إنشاء

.عند التبديل (recreate) وإعادة تحميل النشاط SharedPreferences التفضيل في

نجح مشروع "جذور فلسطين" في تحقيق هدفه الأساسي المتمثل في توحيد وعرض التراث الأثري الفلسطيني ضمن منصة رقمية واحدة، مستقلة، وسهلة الاستخدام.

تم ضمان كود نظيف، قابل للصيانة، MVVM و Clean Architecture من خلال اعتماد معمارية وقابل للتوسع. أثبت التطبيق قدرته على العمل بكفاءة دون اتصال بالإنترنت، مع دعم التعدد اللغوي (عربي/إنجليزي) والوضع الليلي.

يمثل هذا المشروع خطوة عملية نحو رقمنة التراث الفلسطيني، ويعكس المقدرة التقنية على دمج أحدث أدوات تطوير أندرويد مع الهوية الثقافية المحلية، مما يتيح للجيل القادم فرصة التعرف على تاريخه وثقافته بأسلوب تفاعلي ومبتكر.

- [1] Google Developers. (2024). Android Developers Documentation: Kotlin, Room, ViewBinding, Coroutines. developer.android.com
- [2] JetBrains. (2024). Kotlin Programming Language Reference. kotlinlang.org
- [3] Google Maps Platform. (2024). Maps SDK for Android Documentation. developers.google.com/maps
- [4] Material Design. (2024). Material Components for Android Guidelines. m3.material.io
- [5] وزارة السياحة والآثار الفلسطينية. قاعدة بيانات المواقع الأثرية في الضفة الغربية [5]
- [6] والمراجع التاريخية المحلية (UNESCO) مواقع التراث العالمي [6]
- [7] مجتمعات المطورين: Stack Overflow, GitHub, Android Developers Blog.

مساعدة اللغة - LocaleHelper.kt كود

```
package com.palestine.roots.util

import android.content.Context
import android.content.res.Configuration
import java.util.Locale

object LocaleHelper {
    private const val PREFS_NAME = "settings"
    private const val KEY_LANGUAGE = "language"

    fun getSavedLanguage(context: Context): String {
        val prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
        return prefs.getString(KEY_LANGUAGE, "ar") ?: "ar"
    }

    fun applyLocale(context: Context): Context {
        return setLocale(context, getSavedLanguage(context))
    }

    fun setLocale(context: Context, language: String): Context {
        val locale = Locale(language)
        Locale.setDefault(locale)
        val config = Configuration(context.resources.configuration)
        config.setLocale(locale)
        config.setLayoutDirection(locale)
        return context.createConfigurationContext(config)
    }
}
```


إدارة التفضيلات - PreferencesManager.kt كود

```
package com.palestine.roots.data.local

import android.content.Context
import android.content.SharedPreferences
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.callbackFlow
import kotlinx.coroutines.channels.awaitClose

class PreferencesManager(private val context: Context) {
    private val prefs: SharedPreferences =
        context.getSharedPreferences("settings", Context.MODE_PRIVATE)

    val isDarkMode: Flow<Boolean> = callbackFlow {
        val listener = SharedPreferences.OnSharedPreferenceChangeListener { _, key ->
            if (key == KEY_DARK_MODE) trySend(prefs.getBoolean(KEY_DARK_MODE, false))
        }
        prefs.registerOnSharedPreferenceChangeListener(listener)
        trySend(prefs.getBoolean(KEY_DARK_MODE, false))
        awaitClose { prefs.unregisterOnSharedPreferenceChangeListener(listener) }
    }

    val language: Flow<String> = callbackFlow {
        val listener = SharedPreferences.OnSharedPreferenceChangeListener { _, key ->
            if (key == KEY_LANGUAGE) trySend(prefs.getString(KEY_LANGUAGE, "ar") ?: "ar")
        }
        prefs.registerOnSharedPreferenceChangeListener(listener)
        trySend(prefs.getString(KEY_LANGUAGE, "ar") ?: "ar")
        awaitClose { prefs.unregisterOnSharedPreferenceChangeListener(listener) }
    }

    suspend fun setLanguage(lang: String) {
        prefs.edit().putString(KEY_LANGUAGE, lang).apply()
    }
}
```

```
suspend fun toggleDarkMode(enabled: Boolean) {  
    prefs.edit().putBoolean(KEY_DARK_MODE, enabled).apply()  
}  
}
```

كيان قاعدة البيانات - SiteEntity.kt كود

```
package com.palestine.roots.data.local.entity

import androidx.room.ColumnInfo
import androidx.room.Entity
import androidx.room.PrimaryKey
import com.google.gson.annotations.SerializedName

@Entity(tableName = "sites")
data class SiteEntity(
    @PrimaryKey @SerializedName("id") val id: String,
    @SerializedName("name") val name: String,
    @ColumnInfo(name = "name_en") @SerializedName("name_en") val nameEn: String,
    @SerializedName("city") val city: String,
    @ColumnInfo(name = "city_en") @SerializedName("city_en") val cityEn: String,
    @SerializedName("description") val description: String,
    @ColumnInfo(name = "description_en") @SerializedName("description_en") val
descriptionEn: String,
    @SerializedName("history") val history: String,
    @ColumnInfo(name = "history_en") @SerializedName("history_en") val historyEn: String,
    @ColumnInfo(name = "image_url") @SerializedName("imageUrl") val imageUrl: String,
    @SerializedName("latitude") val latitude: Double,
    @SerializedName("longitude") val longitude: Double,
    @SerializedName("category") val category: String,
    @ColumnInfo(name = "category_en") @SerializedName("category_en") val categoryEn: String,
    @ColumnInfo(name = "foundation_year") @SerializedName("foundationYear") val
foundationYear: String?,
    @ColumnInfo(name = "is_favorite", defaultValue = "0") @SerializedName("is_favorite") val
isFavorite: Boolean = false
)
```

نموذج العرض الرئيسي - HomeViewModel.kt كود

```
package com.palestine.roots.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.palestine.roots.data.local.PreferencesManager
import com.palestine.roots.domain.repository.SiteRepository
import kotlinx.coroutines.flow.*

class HomeViewModel(
    private val repository: SiteRepository,
    private val preferencesManager: PreferencesManager
) : ViewModel() {

    private val _uiState = MutableStateFlow<UiState>(UiState.Loading)
    val uiState: StateFlow<UiState> = _uiState.asStateFlow()

    val isDarkMode = preferencesManager.isDarkMode
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), false)

    val language = preferencesManager.language
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), "ar")

    data class Province(val ar: String, val en: String)

    val provinces = listOf(
        Province("القدس", "Jerusalem"),
        Province("نابلس", "Nablus"),
        Province("الخليل", "Hebron"),
        // باقي المحافظات
    )

    fun getProvinceNames(lang: String): List<String> =
        provinces.map { if (lang == "en") it.en else it.ar }
```

```

// ... (loadAllSites, filterByCity, toggleFavorite, setLanguage, toggleDarkMode)

sealed class UiState {
    object Loading : UiState()
    data class Success(val sites: List<Site>) : UiState()
    data class Error(val message: String) : UiState()
}

// Manual DI Factory
class Factory(
    private val repository: SiteRepository,
    private val preferencesManager: PreferencesManager
) : ViewModelProvider.Factory {
    @Suppress("UNCHECKED_CAST")
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        return HomeViewModel(repository, preferencesManager) as T
    }
}

```

النشاط الرئيسي - HomeActivity.kt كود

```
package com.palestine.roots.ui

import android.content.Context
import com.palestine.roots.util.LocaleHelper
import com.palestine.roots.viewmodel.HomeViewModel

class HomeActivity : AppCompatActivity() {

    private val viewModel: HomeViewModel by viewModels {
        val dao = PalestineDatabase.getInstance(this).siteDao()
        val repo = SiteRepositoryImpl(dao)
        val prefs = PreferencesManager(this)
        HomeViewModel.Factory(repo, prefs) // Manual DI
    }

    override fun attachBaseContext(newBase: Context) {
        super.attachBaseContext(LocaleHelper.applyLocale(newBase))
    }

    private fun setupToolbarActions() {
        binding.ibLanguage.setOnClickListener {
            currentLang = if (currentLang == "ar") "en" else "ar"
            viewModel.setLanguage(currentLang)
            LocaleHelper.setLocale(this, currentLang)
            recreate()
        }
        // ... (map, favorites, dark mode actions)
    }
}
```

قاعدة البيانات (مقتطف) - PalestineDatabase.kt كود

```
@Database(entities = [SiteEntity::class], version = 4, exportSchema = false)
abstract class PalestineDatabase : RoomDatabase() {
```

```
abstract fun siteDao(): SiteDao
```

```
companion object {
```

```
    fun getInstance(context: Context): PalestineDatabase {
```

```
        return INSTANCE ?: synchronized(this) {
```

```
            Room.databaseBuilder(
```

```
                context.applicationContext,
```

```
                PalestineDatabase::class.java,
```

```
                "palestine_database"
```

```
            )
```

```
            .fallbackToDestructiveMigration()
```

```
            .addCallback(PopulateCallback(context.applicationContext))
```

```
            .build()
```

```
        }
```

```
    }
```

```
}
```

```
class PopulateCallback(private val context: Context) : RoomDatabase.Callback() {
```

```
    override fun onOpen(db: SupportSQLiteDatabase) {
```

```
        super.onOpen(db)
```

```
        CoroutineScope(Dispatchers.IO).launch { ensureDataPopulated() }
```

```
    }
```

```
    // Uses Gson to parse JSON from assets
```

```
}
```

```
}
```